

Problem 1

Write a class **SingLList**, objects of which represent singly linked lists of **ints**. Inside the class, define a private static class **Node** describing individual nodes of the list — it has fields **data** holding the data (**int**) and **next** of type **Node** pointing to the next node of the list. Such private static inner class is visible only inside the enclosing class (in this case **SingLList**); its fields are also directly visible there — no *getters* are needed. The class **SingLList** itself has only one private field **head** which holds the reference to the first node of the list (possibly **null**, if the list is empty).

The class **SingLList** contains functions:

```
1    public boolean empty()
2    public void addFront(int d)
3    public void addBack(int d)
4    public static SingLList arrayToList(int[] arr)
5    public void removeOdd()
6    public boolean contains(int d)
7    public void showList()
8    public void clear()
```

where

1. method **empty** answers the question if the list is empty;
2. method **addFront** adds a new element with data **d** at the front of the list;
3. method **addBack** adds a new element with data **d** as the last element of the list;
4. static function **arrayToList** takes an array of **ints**, creates and returns a list containing as data the numbers from the array (in the same order!);
5. method **removeOdd** removes from the list all nodes in which **data** is an odd number. If all nodes contain odd numbers, they all should be removed and the list should now be empty. The method cannot create any new nodes;
6. method **contains** answers the question if the list contains an element with data equal to **d**;
7. method **showList** prints the content of the list (data from all nodes, in one line, separated by spaces);
8. method **clear** makes the list empty.

The following **main** function:

```
public static void main(String[] args) {
    int[] arr = {1,2,3,4,5,6,7};
    SingLList list = SingLList.arrayToList(arr);
    list.showList();
    list.removeOdd();
}
```

[download SingLList.java](#)

```

        list.showList();
        list.addFront(1);
        list.addBack(8);
        list.showList();
        System.out.println("contains 3? "+list.contains(3));
        System.out.println("contains 8? "+list.contains(8));
        list.clear();
        list.showList();
    }

```

should print

```

1 2 3 4 5 6 7
2 4 6
1 2 4 6 8
contains 3? false
contains 8? true
Empty list

```

Problem 2

Create a class **QueueOfTasks** objects of which represent queues (FIFO) of elements of type **Task**. The implementation should be based on singly-linked list, with nodes being objects of the **Node** class with fields **task** of type **Task** and **next** of type **Node** (pointing to the next node).

The class **QueueOfTasks** should provide, as any decent queue does, three methods:

- method **public void enqueue**(Task t) adding new object of type **Node** at the end (tail) of the queue;
- method **public Task dequeue**() removing the node from the front (head) of the queue and returning the data task from this node;
- method **public boolean empty**() returning **true** if and only if the queue is empty.

Note that a queue is basically a singly-linked list where adding is always performed at the back end of the list, while removing at its front end. Therefore, if class **QueueOfTasks** has field **tail** pointing to the last node and **head** pointing to the first, no iteration over the list will ever be necessary.

The **Task** class represents a task: it contains input data (in our case two numbers) and the **execute** method which somehow processes this data and returns a result (here, just a number of type **double**):

```

public class Task {
    double x, y;
    protected Task(double a, double b) {
        x = a;
        y = b;
    }
}

```

```

    public double execute() {
        throw new UnsupportedOperationException();
    }
    public static Task getInstance(String oper,
                                   double a, double b) {
        // ...
    }
}

```

The **execute** function just throws an exception terminating the job (as shown above); however, there are classes extending the **Task** class where it is overridden and provides various implementations of processing the data: **Adder**, **Multiplier**, **Divider** and **Subtractor**. The static function **getInstance** returns objects of these classes, selected according to the value of the string **oper**.

The program reads lines from the console (using **Scanner** or **Console** — see the documentation — note that **Console** may not work under an IDE). Each line specifies the action to be performed (as one of the symbols '+', '-', '/', '*') and input data for one task:

```

import java.util.Locale;
import java.util.Scanner;

public class TaskQueue {
    public static void main(String[] args) {
        // we want decimal dot, not a comma...
        Locale.setDefault(Locale.US);
        QueueOfTasks queue = new QueueOfTasks();
        Scanner console = new Scanner(System.in);
        String op = null;
        System.out.println(
            "Enter tasks in the form 'op arg1 arg2'\n" +
            "where op is + - * / and args are doubles\n" +
            "Enter 'q' to stop reading");
        while (!(op = console.next()).startsWith("q")) {
            double a = console.nextDouble();
            double b = console.nextDouble();
            Task task = Task.getInstance(op, a, b);
            queue.enqueue(task);
        }
        while (!queue.empty()) {
            Task task = queue.dequeue();
            System.out.println(task.execute());
        }
    }
}

```

[download TaskQueue.java](#)

For example, execution of the program may look like this:

```

Enter tasks in the form 'op arg1 arg2'
where op is + - * / and args are doubles
Enter 'q' to stop reading
+ 3 9
/ 24 6
- 90.5 100.5
* -3 -12
q
12.0
4.0
-10.0
36.0

```

Don't use arrays or any classes from the *java.util* package!

Problem 3

Create two classes

- **Person** representing a person with fields **name** of type **String** and **cars** of type **Car** representing cars which they owns; this can be **null** if he/she doesn't have any car;
- **Car** representing a car with fields **make** (**String**), **price** (**int**) and **next** of type **Car** which is the reference to the next car possessed by a person (possibly **null**, then *this* car is the last). In other words, objects of class **Car** can be viewed as nodes of a singly-linked list.

All fields in both classes should be **private**!

The class **Car** contains the following member functions (constructors and methods):

```

1  public Car(String m, int p, Car n) { ... }
2  public Car(String make, int price) { ... }
3  public String getMake() { ... }
4  public int getPrice() { ... }
5  public Car getNext() { ... }
6  public void showCars() { ... }
7  public void showCarsRev() { ... }
8  @Override
9  public String toString() { ... }

```

where

1. the first two member functions are constructors: one taking values of all fields, and one taking a make and a price and setting **next** to **null**;
2. the next three methods are just accessors which return values of the corresponding fields;
3. **showCars** prints (using **System.out.print** – see below) information on all cars: *this* car, then the one referenced to by its field **next**, then next of this next e.t.c, until **null** is encountered. Information on consecutive cars should be printed in one line;

4. **showCarsRev** prints (using **System.out.print** – see below) information on all cars, as does **showCars**, but in the reverse order: it has to be recursive and must not use any loops, auxiliary arrays or additional fields;
5. **toString** overrides **toString** from class **Object** and returns a string with make and price of *this* car. In this way a reference to **Car** may be used as an argument to **System.out.print** yielding a sensible string.

The class **Person** contains the following member functions (constructors and methods):

```

1  public Person(String name) { ... }
2  public Person buys(String make, int price) { ... }
3  public String getName() { ... }
4  public void showCars() { ... }
5  public void showCarsRev() { ... }
6  public int getTotalPrice() { ... }
7  public boolean hasCar(String make) { ... }
8  public Car mostExpensive() { ... }

```

where

1. the first member is the constructor taking **name** only; field **cars** will be initialized with **null**;
2. **buys** takes a **String** and an **int**, creates a **Car** with the given **make** and **price** and sets it as the first car possessed by *this* person; the car which was the first (possibly **null**) becomes the second, i.e., it is referenced by the field **next** of the newly created car. The method returns the reference to *this* person (the one the method has been invoked on);
3. **getName** is a simple getter method;
4. **showCars** prints all cars owned by *this* person using the corresponding method in class **Car**;
5. **showCarsRev** does the same thing, but utilizing the corresponding recursive method in **Car**;
6. **getTotalPrice** returns the total price of all cars owned by *this* person;
7. **hasCar** takes a **String** and returns a **boolean** stating if *this* person owns a car of the given make (use **equalsIgnoreCase** from **String** to make string comparison case insensitive);
8. **mostExpensive** returns the most expensive car owned by *this* person (or **null** if *this* person doesn't have a car).

The following **main** function (in another class)

```

public static void main (String[] args) {
    Person john = new Person("John");
    john.buys("Ford", 20000)
        .buys("Opel", 16000)
        .buys("Fiat", 12000)

```

[download PersonCars.java](#)

```

        .showCars();
        System.out.println();

        john.showCarsRev();
        System.out.println();

        System.out.println("Total price of " +
            john.getName() + "'s cars: " +
            john.getTotalPrice());
        System.out.println("Does " + john.getName() +
            " have a ford? " + john.hasCar("ford"));
        System.out.println("Does " + john.getName() +
            " have a bmw? " + john.hasCar("bmw"));
        System.out.println(john.getName() + "'s most " +
            "expensive car is " + john.mostExpensive());
    }

```

should print

```

Fiat(12000) Opel(16000) Ford(20000)
Ford(20000) Opel(16000) Fiat(12000)
Total price of John's cars: 48000
Does John have a ford? true
Does John have a bmw? false
John's most expensive car is Ford(20000)

```
